AI-Powered Smart Shopping Cart System with IoT Connections

Project Abstract and Report

1. Abstract

In most shops, customers must wait in a long line at the billing counter while each item is scanned by hand. This project builds a **smart shopping cart** that removes this line. The cart knows which item the customer puts in, keeps a running total on its screen, and charges the customer automatically when they leave the shop.

The cart uses two sensors together. A **camera** with a YOLOv8 model recognises *what* the item is. A **load cell** with an HX711 amplifier measures the *weight* change to confirm that the item was really added or removed. When both sensors agree, the cart sends the event to the cloud using the MQTT or WebSocket protocol. A database gives the price, the total is updated on the cart screen, and at the exit the bill is paid automatically through Razorpay and UPI Autopay. Using two sensors instead of one makes the bill more accurate.

Keywords: smart shopping cart, IoT, computer vision, YOLOv8, HX711 load cell, sensor fusion, MQTT, UPI Autopay.

2. Introduction

Waiting at the billing counter wastes time for customers and needs extra staff for the shop. Manual scanning can also cause mistakes. Some large companies already make smart carts, such as Amazon Dash Cart and Caper Cart, but they are expensive. This project aims to build a simple and low-cost version using common hardware and free software, and to support UPI, which is the most common digital payment method in India.

2.1 Objectives

- Recognise each product that the customer puts in the cart.
- Confirm every item using weight, so that wrong bills are avoided.
- Tell the difference between adding an item and putting it back.
- Show the running total on the cart screen at all times.
- Take payment automatically when the customer leaves.

3. System Overview

The system works in eight stages, from picking up the cart to the final payment. Figure 1 shows the complete workflow. Stages 2 to 6 happen in the processing core, where the item is recognised, checked and sent to the cloud.

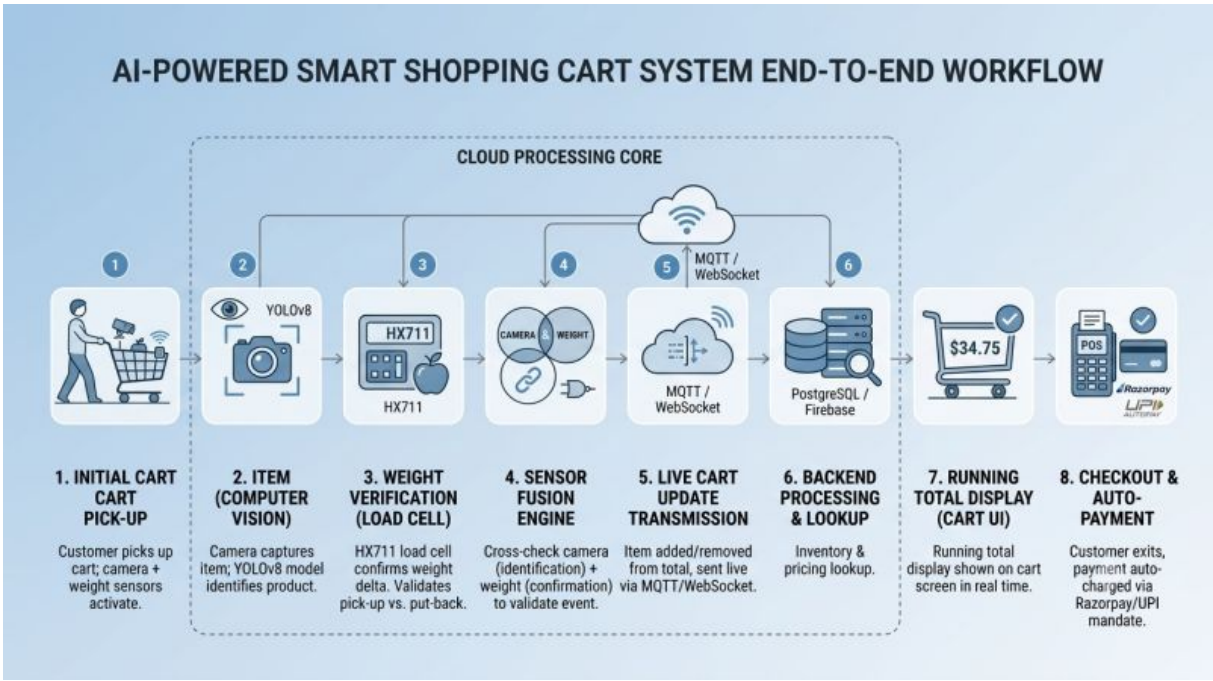


Figure 1. End-to-end workflow of the smart shopping cart.

4. How the System Works

4.1 Stage 1: Cart pick-up

The customer takes a cart. The camera and weight sensors switch on and a new shopping session starts.

4.2 Stage 2: Item recognition

When an item is placed in the cart, the camera takes pictures. The YOLOv8 model, trained on pictures of the shop's products, finds the item and tells which product it is.

4.3 Stage 3: Weight check

The load cell sits under the cart basket. The HX711 amplifier turns its tiny signal into a number the computer can read. If the weight goes up, an item was added. If it goes down, an item was removed.

4.4 Stage 4: Sensor fusion

The system compares the camera result with the weight result. The item is accepted only if both agree. For example, if the camera sees a 500 g packet and the weight rises by about 500 g, the item is added to the bill.

4.5 Stage 5: Sending the update

The cart sends the confirmed event to the cloud using MQTT or WebSocket. These are light and fast methods, made for small IoT devices.

4.6 Stage 6: Price lookup

The backend server finds the price of the product in the database (PostgreSQL or Firebase) and updates the stock count.

4.7 Stage 7: Running total

The new total is sent back and shown on the cart screen, so the customer always knows how much they will pay.

4.8 Stage 8: Checkout and payment

When the customer leaves the shop, the final amount is charged automatically through Razorpay using a UPI Autopay mandate. No billing counter is needed.

5. Flowchart of the Cart Logic

Figure 2 shows how the cart decides what to do each time the customer adds or removes an item. If the camera is not sure, or if the weight does not match, the cart does not add the item to the bill. It asks the customer to show the item again or marks the event for review.

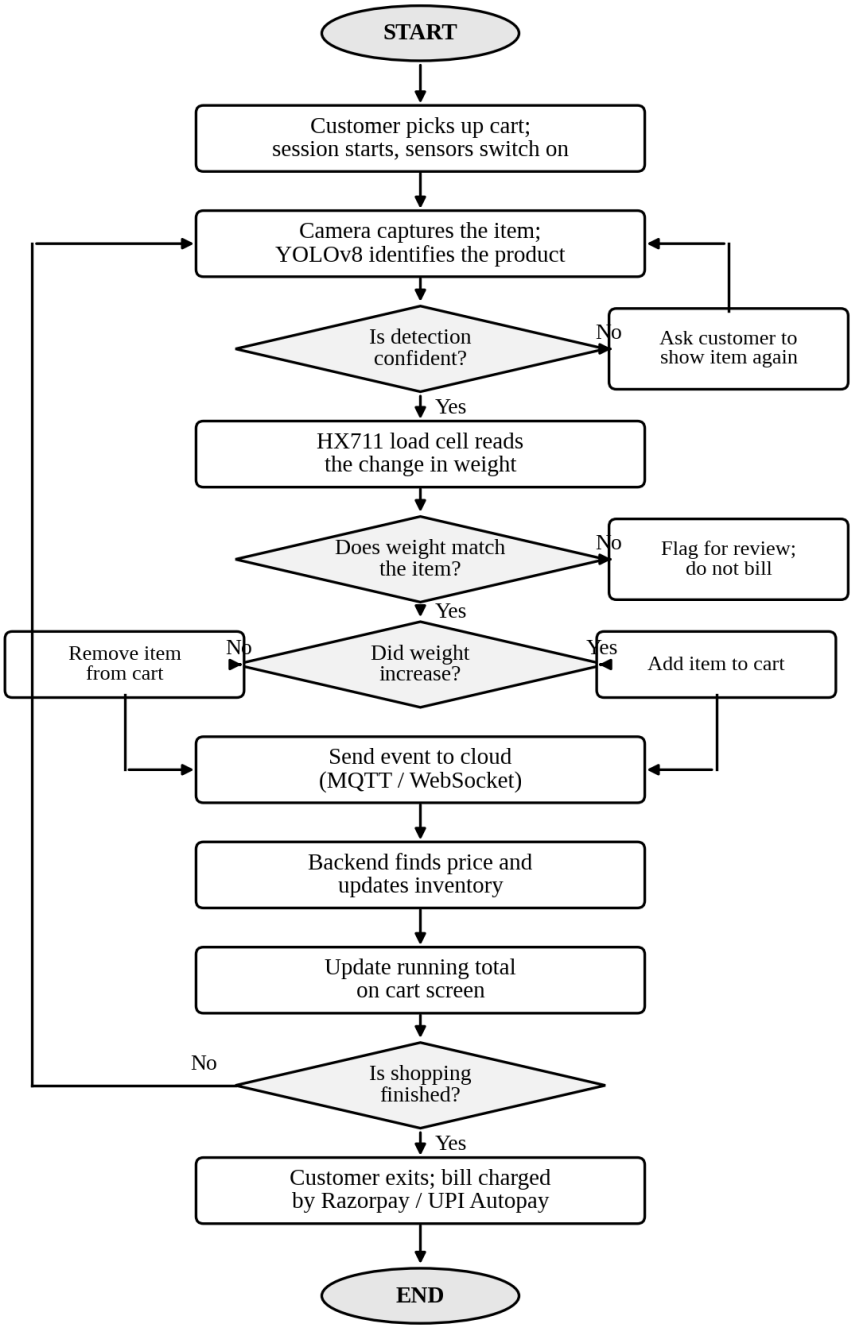


Figure 2. Flowchart of the item detection and billing logic.

6. IoT Tools and How to Connect Them

An IoT (Internet of Things) project joins small hardware devices to software over a network. In this cart, the sensors connect to a small computer on the cart, which connects to the cloud through Wi-Fi. Figure 3 shows how all the parts are joined.

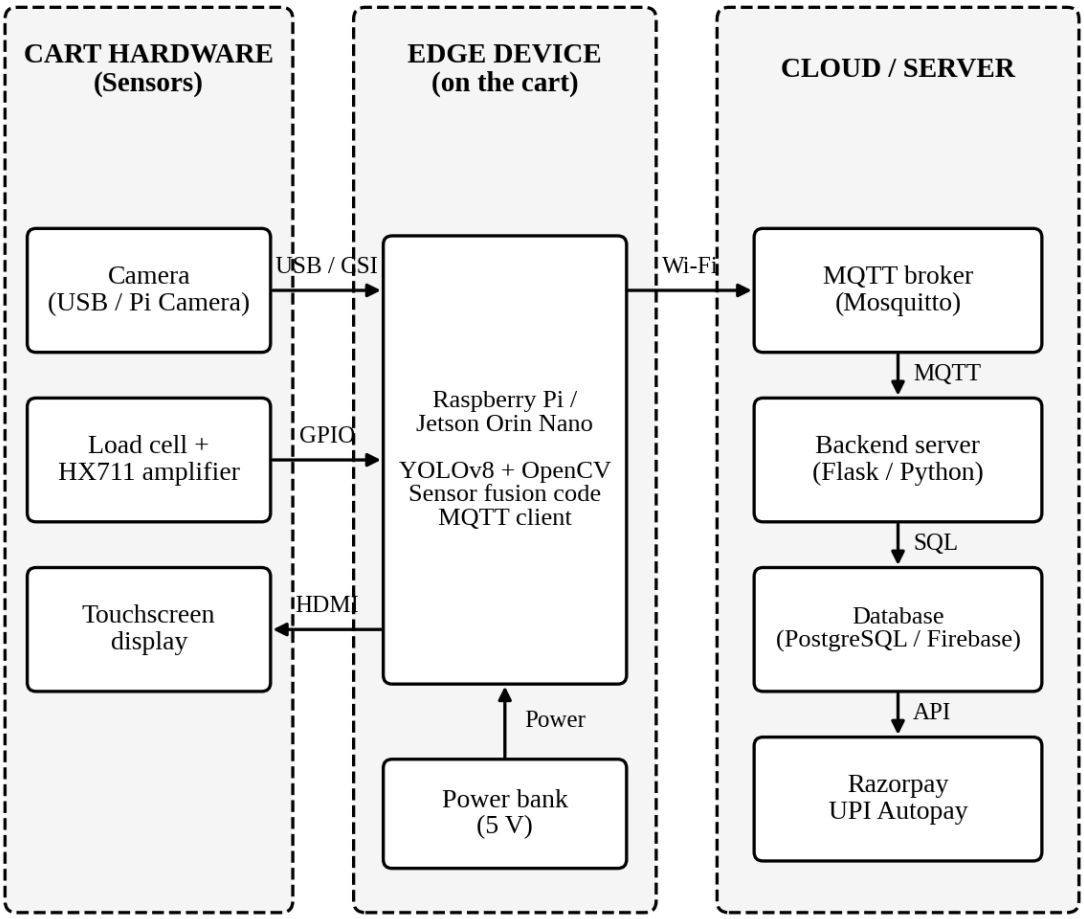


Figure 3. Connection diagram of the IoT hardware and cloud.

6.1 Hardware used

Component	What it does	How it connects
Raspberry Pi 4 or Jetson Orin Nano	Small computer on the cart. Runs the camera model and the fusion code.	Main board; all sensors connect to it
Camera (USB webcam or Pi Camera)	Takes pictures of items.	USB port or camera (CSI) port
Load cell	Senses the weight in the cart.	Wires to the HX711 board
HX711 amplifier	Makes the small load cell signal readable.	Two data wires to Raspberry Pi GPIO pins
Touchscreen display	Shows the item list and total.	HDMI or DSI port
Power bank (5 V, 3 A)	Powers the cart.	USB-C power input of the Pi
Wi-Fi	Sends data to the cloud.	Built into the Raspberry Pi

Table 1. Hardware components.

6.2 Connecting the HX711 to the Raspberry Pi

The HX711 is the most important connection, because it gives the weight data. The usual wiring for a Raspberry Pi is shown in Table 2. Any free GPIO pins can be used, so the pin numbers below are only an example.

HX711 pin	Connect to	Note
VCC	Pi 3.3 V (pin 1)	3.3 V keeps the data signal safe for the Pi
GND	Pi GND (pin 6)	Common ground
DT (data)	Pi GPIO 5 (pin 29)	Sends weight data
SCK (clock)	Pi GPIO 6 (pin 31)	Clock signal from the Pi
E+ / E-	Load cell red / black wire	Power for the load cell
A+ / A-	Load cell green / white wire	Signal from the load cell

Table 2. HX711 wiring (example). Wire colours can differ between load cell makers, so check the datasheet before connecting.

6.3 Software tools

Tool	Use in the project
Python 3	Main programming language on the cart and the server
Ultralytics YOLOv8 and OpenCV	Camera input and product detection
HX711 Python library	Reads weight values from the load cell
paho-mqtt	Sends cart events from the Pi to the broker
Mosquitto (MQTT broker)	Receives messages from all carts and passes them on
Flask	Backend server that handles events and prices
PostgreSQL or Firebase	Stores products, prices, stock and sessions
Razorpay API with UPI Autopay	Automatic payment at exit
Roboflow or CVAT	Labelling product images to train YOLOv8

Table 3. Software tools.

6.4 Message sent by the cart

Each event is sent to the MQTT broker on a topic such as *cart/01/events*. The message is small and contains the fields in Table 4.

Field	Example	Meaning
cart_id	01	Which cart sent the message
item	Apple	Product found by the camera
action	add	Add or remove, found from the weight change
weight_g	182	Measured weight change in grams
confidence	0.93	How sure the camera model is

Table 4. Example fields of one cart event.

7. Sensor Fusion Decisions

Neither sensor is trusted alone. The camera may confuse similar items, and the load cell can be disturbed when the cart moves. Using both together removes most of these errors.

Camera	Weight	Decision
Item found, confident	Weight rises as expected	Add item and update total
Item found, confident	Weight falls as expected	Remove item (put-back)
Item found	Weight does not match	Do not bill; flag for review
Not sure	Weight changes	Do not bill; ask customer to show item again
Item seen	No weight change	Ignore (item was not placed in the cart)

Table 5. Examples of fusion decisions.

8. Advantages

- No waiting in the billing line.
- More accurate bills because two sensors check each other.
- The customer sees the total in real time.
- The shop gets automatic stock updates.
- Works with UPI, which most Indian customers already use.
- Low cost, because it uses common parts and free software.

9. Challenges and Future Work

Some problems must be solved before real use: items hidden under other items, products with similar packages, noise in the weight reading when the cart moves, store lighting, and loose goods such as vegetables. Future work includes building a large labelled picture dataset of store products, adding object tracking and filtering for the weight signal, and testing newer detection models such as RF-DETR. This is also important because the YOLOv8 licence (AGPL) can be a problem for commercial products. Finally, a small trial in a real shop is needed to measure accuracy, speed and customer experience.

10. Conclusion

This project shows how a camera, a load cell, a small computer, cloud messaging and digital payment can work together as a checkout-free shopping cart. Its main idea is sensor fusion: every item must be confirmed by both the camera and the weight sensor before it is billed. With more testing and a real-shop trial, the system can become a practical and affordable automated shopping solution.